

Great 🚀 Let's dive into **Module 20: Stored Procedures**.

SQL CLASS NOTE – MODULE 20: STORED PROCEDURES

1. Introduction

A **Stored Procedure (SP)** is a **saved SQL program** stored in the database.

- You can call it whenever you need it.
- It may include:
 - SELECT , INSERT , UPDATE , DELETE statements
 - Conditions (IF , CASE)
 - Loops
 - Transactions

👉 Stored Procedures make SQL code **reusable, modular, and secure**.

2. Syntax

Create a Stored Procedure

```
DELIMITER $$
```

```
CREATE PROCEDURE procedure_name (parameters)
```

```
BEGIN
```

```
-- SQL statements
```

```
END $$
```

```
DELIMITER ;
```

Call a Stored Procedure

```
CALL procedure_name(arguments);
```

Drop a Stored Procedure

```
DROP PROCEDURE procedure_name;
```

3. Types of Parameters

1. **IN** – Input parameter (default).
2. **OUT** – Returns a value from the procedure.
3. **INOUT** – Works as both input and output.

4. Practical Examples with SchoolDB

Example 1: Simple procedure to show all students

```
DELIMITER $$
```

```
CREATE PROCEDURE GetAllStudents()
```

```
BEGIN
```

```
    SELECT * FROM Students;
```

```
END $$
```

```
DELIMITER ;
```

Usage:

```
CALL GetAllStudents();
```

Example 2: Procedure with input parameter (show students by grade)

```
DELIMITER $$
```

```
CREATE PROCEDURE GetStudentsByGrade(IN grade VARCHAR(20))
```

```
BEGIN
```

```
    SELECT StudentID, FirstName, LastName, Age
```

```
    FROM Students
```

```
    WHERE GradeLevel = grade;
```

```
END $$
```

```
DELIMITER ;
```

Usage:

```
CALL GetStudentsByGrade('Grade 12');
```

Example 3: Procedure with multiple parameters (insert a new student)

```
DELIMITER $$
```

```
CREATE PROCEDURE AddStudent(
```

```
    IN sid INT, IN fname VARCHAR(50), IN lname VARCHAR(50),
```

```
    IN age INT, IN gender VARCHAR(10), IN grade VARCHAR(20)
```

```
)
```

```
BEGIN
```

```
    INSERT INTO Students (StudentID, FirstName, LastName, Age, Gender, GradeLevel)
```

```
    VALUES (sid, fname, lname, age, gender, grade);
```

```
END $$
```

```
DELIMITER ;
```

Usage:

```
CALL AddStudent(210, 'James', 'Bond', 17, 'Male', 'Grade 11');
```

Example 4: Procedure with output parameter (count students in a grade)

```
DELIMITER $$

CREATE PROCEDURE CountStudentsByGrade(
    IN grade VARCHAR(20),
    OUT total INT
)
BEGIN
    SELECT COUNT(*) INTO total
    FROM Students
    WHERE GradeLevel = grade;
END $$

DELIMITER ;
```

Usage:

```
CALL CountStudentsByGrade('Grade 10', @total);
SELECT @total;
```

Example 5: Procedure with transaction (enroll student in a course)

```
DELIMITER $$

CREATE PROCEDURE EnrollStudent(
    IN sid INT, IN cid INT, IN eid INT
)
BEGIN
    START TRANSACTION;
    INSERT INTO Enrollments (EnrollmentID, StudentID, CourseID)
    VALUES (eid, sid, cid);
    COMMIT;
END $$

DELIMITER ;
```

Usage:

```
CALL EnrollStudent(101, 103, 700);
```

5. Why Use Stored Procedures?

-  Reuse logic (no need to rewrite queries).
-  Improved **performance** (compiled and stored in DB).
-  Better **security** (control user access via procedures).
-  Maintainability (change logic in one place).

6. Real-Life Use Cases

- **Schools:** SP to add a new student and auto-enroll them.
- **Banks:** SP to transfer money between accounts.
- **E-commerce:** SP to process orders and update inventory.
- **Hospitals:** SP to admit a patient and assign a doctor.

7. Classwork (Practice)

1. Create a procedure `GetAllStudents` to display all students.
2. Create a procedure `GetStudentsByGrade` to list students in a grade.
3. Create a procedure `AddStudent` that inserts a new student.
4. Create a procedure `CountStudentsByGrade` that returns the number of students in a grade.
5. Create a procedure `EnrollStudent` that inserts into `Enrollments` using a transaction.
6. Call all the above procedures with real arguments.

8. Challenge Question

👉 Create a procedure called `TransferCredits` that:

- Takes `from_student`, `to_student`, and `amount`.
- Deducts credits from one student.
- Adds credits to another.

- Uses a **transaction** so both succeed or none.

✅ That's the **expanded Module 20: Stored Procedures**.

Perfect 👍 Let's solve the **Module 20 Classwork + Challenge** with SQL code examples.

✅ Solutions to Module 20: STORED PROCEDURES

1. Create a procedure `GetAllStudents` to display all students.

```
DELIMITER $$
```

```
CREATE PROCEDURE GetAllStudents()
```

```
BEGIN
```

```
    SELECT * FROM Students;
```

```
END $$
```

```
DELIMITER ;
```

```
-- Call it
```

```
CALL GetAllStudents();
```

2. Create a procedure `GetStudentsByGrade` to list students in a grade.

```
DELIMITER $$

CREATE PROCEDURE GetStudentsByGrade(IN grade VARCHAR(20))
BEGIN
    SELECT StudentID, FirstName, LastName, Age
    FROM Students
    WHERE GradeLevel = grade;
END $$

DELIMITER ;

-- Call it
CALL GetStudentsByGrade('Grade 11');
```

3. Create a procedure `AddStudent` that inserts a new student.

```
DELIMITER $$

CREATE PROCEDURE AddStudent(
    IN sid INT, IN fname VARCHAR(50), IN lname VARCHAR(50),
    IN age INT, IN gender VARCHAR(10), IN grade VARCHAR(20)
)
BEGIN
    INSERT INTO Students (StudentID, FirstName, LastName, Age, Gender, GradeLevel)
    VALUES (sid, fname, lname, age, gender, grade);
END $$

DELIMITER ;

-- Call it
CALL AddStudent(211, 'Sophia', 'Green', 15, 'Female', 'Grade 9');
```

4. Create a procedure `CountStudentsByGrade` that returns the number of students in a grade.

```
DELIMITER $$

CREATE PROCEDURE CountStudentsByGrade(
    IN grade VARCHAR(20),
    OUT total INT
)
BEGIN
    SELECT COUNT(*) INTO total
    FROM Students
    WHERE GradeLevel = grade;
END $$

DELIMITER ;

-- Call it
CALL CountStudentsByGrade('Grade 10', @total);
SELECT @total;
```

5. Create a procedure `EnrollStudent` that inserts into `Enrollments` using a transaction.

```
DELIMITER $$

CREATE PROCEDURE EnrollStudent(
    IN eid INT, IN sid INT, IN cid INT
)
BEGIN
    START TRANSACTION;
    INSERT INTO Enrollments (EnrollmentID, StudentID, CourseID)
    VALUES (eid, sid, cid);
    COMMIT;
END $$

DELIMITER ;

-- Call it
CALL EnrollStudent(701, 101, 104);
```




Challenge Question

👉 Create a procedure `TransferCredits` that transfers credits between students.

```
DELIMITER $$
```

```
CREATE PROCEDURE TransferCredits(  
    IN from_student INT,  
    IN to_student INT,  
    IN amount INT  
)  
BEGIN  
    START TRANSACTION;  
  
    UPDATE Students  
    SET Credits = Credits - amount  
    WHERE StudentID = from_student;  
  
    UPDATE Students  
    SET Credits = Credits + amount  
    WHERE StudentID = to_student;  
  
    COMMIT;  
END $$
```

```
DELIMITER ;
```

```
-- Call it  
CALL TransferCredits(101, 102, 10);
```

📌 This ensures that credits are either fully transferred or not transferred at all (atomic).